In [3]:
```
import nltk
import pandas as pd

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag

from sklearn.feature_extraction.text import TfidfVectorizer
```

In [12]:
```
nltk.download('punkt_tab')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package punkt_tab to
[nltk_data]     C:\Users\user\AppData\Roaming\nltk_data...
[nltk_data]   Package punkt_tab is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\user\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to
[nltk_data]     C:\Users\user\AppData\Roaming\nltk_data...
[nltk_data]   Package wordnet is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data]     C:\Users\user\AppData\Roaming\nltk_data...
[nltk_data]   Unzipping taggers\averaged_perceptron_tagger_eng.zip.
```

Out[12]: True

1. nltk.download('punkt'): Downloads the Punkt sentence tokenizer, which is used for dividing text into a list of sentences or words (tokenization).
2. nltk.download('stopwords'): Downloads the list of common stop words (e.g., "the", "is", "in") for various languages, which are typically removed during text preprocessing.
3. nltk.download('wordnet'): Downloads the WordNet lexical database, a large database of semantic relationships between words (synonyms, antonyms, etc.) used for lemmatization.
4. nltk.download('averaged_perceptron_tagger'): Downloads the trained model used for Part-of-Speech (POS) tagging, which identifies grammatical components like nouns, verbs, and adjectives

In [13]:
```
document = "Natural Language Processing is very intresting and useful for test anal
tokens = word_tokenize(document)
print(tokens)
```

```
['Natural', 'Language', 'Processing', 'is', 'very', 'intresting', 'and', 'useful',
'for', 'test', 'analytics', '.']
```

In [14]:
```
pos = pos_tag(tokens)
```

```
print(pos)
```

```
[('Natural', 'JJ'), ('Language', 'NNP'), ('Processing', 'NNP'), ('is', 'VBZ'), ('ver
y', 'RB'), ('intresting', 'JJ'), ('and', 'CC'), ('useful', 'JJ'), ('for', 'IN'), ('t
est', 'NN'), ('analytics', 'NNS'), ('.', '.')]
```

Nouns (N)

1. NN: Noun, singular or mass (e.g., dog, air, water).
2. NNS: Noun, plural (e.g., dogs, files).
3. NNP: Proper noun, singular (e.g., London, John).
4. NNPS: Proper noun, plural (e.g., Americans, Vikings).

Verbs (V)

1. VB: Verb, base form (e.g., eat, go).
2. VBD: Verb, past tense (e.g., ate, went).
3. VBG: Verb, gerund or present participle (e.g., eating, going).
4. VBN: Verb, past participle (e.g., eaten, gone).
5. VBP: Verb, non-3rd person singular present (e.g., eat, go).
6. VBZ: Verb, 3rd person singular present (e.g., eats, goes).

Adjectives & Adverbs (J, R)

1. JJ: Adjective (e.g., big, fast).
2. JJR: Adjective, comparative (e.g., bigger, faster).
3. JJS: Adjective, superlative (e.g., biggest, fastest).
4. RB: Adverb (e.g., quickly, fast).
5. RBR: Adverb, comparative (e.g., faster, better).
6. RBS: Adverb, superlative (e.g., fastest, best).

Pronouns & Determiners (P, D)

1. PRP: Personal pronoun (e.g., I, you, he, she, it).
2. PRP$: Possessive pronoun (e.g., my, your, his, hers).
3. DT: Determiner (e.g., the, a, these, those).
4. WP: WH-pronoun (e.g., who, what, which).

In [15]:
```
stop_words = set(stopwords.words('english'))

filtered_words = []

for word in tokens:
    if word.lower() not in stop_words:
        filtered_words.append(word)

print(filtered_words)
```

```
['Natural', 'Language', 'Processing', 'intresting', 'useful', 'test', 'analytics',
'.']
```

In [16]:
```
ps = PorterStemmer()

stemmed_words = []

for word in filtered_words:
    stemmed_words.append(ps.stem(word))

print(stemmed_words)
```

```
['natur', 'languag', 'process', 'intrest', 'use', 'test', 'analyt', '.']
```

In [17]:
```
lemmatizer = WordNetLemmatizer()

lemmatized_words = []

for word in filtered_words:
    lemmatized_words.append(lemmatizer.lemmatize(word))

print(lemmatized_words)
```

```
['Natural', 'Language', 'Processing', 'intresting', 'useful', 'test', 'analytics',
'.']
```

In [18]:
```
documents = [
    "Natural language processing is interesting",
    "Text analytics uses natural language processing",
    "Machine learning is useful for analytics"
]

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())

print(X.toarray())
```

```
['analytics' 'for' 'interesting' 'is' 'language' 'learning' 'machine'
 'natural' 'processing' 'text' 'useful' 'uses']
[[0.         0.         0.54935123 0.41779577 0.41779577 0.
  0.         0.41779577 0.41779577 0.         0.         0.        ]
 [0.36617957 0.         0.         0.         0.36617957 0.
  0.         0.36617957 0.36617957 0.48148213 0.         0.48148213]
 [0.3349067  0.44036207 0.         0.3349067  0.         0.44036207
  0.44036207 0.         0.         0.         0.44036207 0.        ]]
```

DSBDA Practical 7 — Text Analytics

NLP Preprocessing, TF and IDF

Aim

To perform:

- Tokenization
- POS Tagging
- Stopword Removal
- Stemming
- Lemmatization
- Term Frequency (TF)
- Inverse Document Frequency (IDF)

using Python.

What is Text Analytics?

Text Analytics (NLP — Natural Language Processing) is used to:

- Process text data
- Extract useful information
- Analyze language

Applications:

- Chatbots
- Spam filtering
- Sentiment analysis
- Search engines

Sample Document

We use sample text:

```
document = "Natural Language Processing is very interesting and useful for text analytics."
```

Step 1: Import Libraries

Code

```
import nltk
import pandas as pd

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag

from sklearn.feature_extraction.text import TfidfVectorizer
```

Explanation

Library	Purpose
nltk	NLP operations
word_tokenize	Tokenization
stopwords	Remove common words
PorterStemmer	Stemming
WordNetLemmatizer	Lemmatization
pos_tag	POS tagging
TfidfVectorizer	TF-IDF calculation

Step 2: Download NLTK Resources

Code

```
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
```

Explanation

Downloads required NLP datasets.

Viva Questions

Why nltk.download() needed?

To download NLP language resources.

Step 3: Tokenization

What is Tokenization?

Breaking text into smaller units called:

`tokens`

Code

```
document = "Natural Language Processing is very interesting and useful for text analytics."

tokens = word_tokenize(document)

print(tokens)
```

Output

```
['Natural', 'Language', 'Processing', 'is', 'very',
 'interesting', 'and', 'useful', 'for', 'text', 'analytics', '.']
```

Explanation

`word_tokenize()`

Splits sentence into words.

Viva Questions

What is tokenization?

Process of splitting text into tokens.

Types of tokenization?

Type	Example
Word tokenization	Split into words
Sentence tokenization	Split into sentences

Step 4: POS Tagging

What is POS Tagging?

Assigning grammatical tags:

- Noun
- Verb
- Adjective

etc.

Code

```
pos = pos_tag(tokens)

print(pos)
```

Output Example

```
[('Natural', 'JJ'),
 ('Language', 'NN'),
 ('Processing', 'NN'),
 ('is', 'VBZ')]
```

Common POS Tags

Tag	Meaning
NN	Noun
VB	Verb
JJ	Adjective

Viva Questions

What is POS tagging?

Assigning grammatical category to words.

Why POS tagging useful?

Helps understand sentence structure.

Step 5: Stopword Removal

What are Stopwords?

Common words with little meaning:

- is
- the
- and
- for

Code

```
stop_words = set(stopwords.words('english'))

filtered_words = []

for word in tokens:
    if word.lower() not in stop_words:
        filtered_words.append(word)
```

```
print(filtered_words)
```

Output

```
['Natural', 'Language', 'Processing',
 'interesting', 'useful', 'text', 'analytics', '.']
```

Explanation

Removes unnecessary words.

Viva Questions

Why remove stopwords?

To reduce noise and improve efficiency.

Step 6: Stemming

What is Stemming?

Reducing word to root form.

Example:

```
playing → play
```

Code

```
ps = PorterStemmer()

stemmed_words = []

for word in filtered_words:
```

```
    stemmed_words.append(ps.stem(word))

print(stemmed_words)
```

Output Example

```
['natur', 'languag', 'process', 'interest', 'use']
```

Explanation

`PorterStemmer()`

Applies stemming rules.

Problem with Stemming

May produce:

`non-dictionary words`
Example:

`studies → studi`

Viva Questions

What is stemming?

Process of reducing words to root form.

Disadvantage of stemming?

May produce meaningless roots.

Step 7: Lemmatization

What is Lemmatization?

Converts word into meaningful dictionary root.

Example:

`better → good`

Code

```
lemmatizer = WordNetLemmatizer()

lemmatized_words = []

for word in filtered_words:
    lemmatized_words.append(lemmatizer.lemmatize(word))

print(lemmatized_words)
```

Output Example

```
['Natural', 'Language', 'Processing',
 'interesting', 'useful']
```

Difference Between Stemming and Lemmatization

Stemming	Lemmatization
Faster	Slower
Rule based	Dictionary based
May produce invalid words	Produces meaningful words

Viva Questions

Which is better: stemming or lemmatization?

Lemmatization gives meaningful roots.

Step 8: Term Frequency (TF)

What is TF?

Measures:

- How frequently a term appears in document

TF Formula

$$TF = \frac{Number\ of\ times\ term\ appears}{Total\ number\ of\ terms}$$

Step 9: Inverse Document Frequency (IDF)

What is IDF?

Measures importance of term across documents.

Rare words:

- Higher IDF

Common words:

- Lower IDF

IDF Formula

$$IDF = \log\left(\frac{Total\ Documents}{Documents\ containing\ term}\right)$$

Step 10: TF-IDF Representation

Code

```
documents = [
    "Natural language processing is interesting",
    "Text analytics uses natural language processing",
    "Machine learning is useful for analytics"
]

vectorizer = TfidfVectorizer()

X = vectorizer.fit_transform(documents)

print(vectorizer.get_feature_names_out())

print(X.toarray())
```

Explanation

`TfidfVectorizer()`

Calculates:

- TF
- IDF
- TF-IDF score

Output

Feature names:

```
['analytics' 'interesting' 'language' ...]
```

Matrix values represent:

```
TF-IDF scores
```

What Does TF-IDF Do?

Highlights important words.

Words occurring frequently in one document but rarely overall get:

```
higher score
```

Important Theory

What is NLP?

Natural Language Processing enables computers to understand human language.

Applications of NLP

- Chatbots
- Translation
- Speech recognition
- Spam filtering

What is Corpus?

Collection of documents/texts.

What is Vocabulary?

Unique words in corpus.

Difference Between TF and IDF

TF	IDF
Frequency in document	Importance across documents

Why TF-IDF Better Than Simple Count?

Because:

- Reduces importance of common words
- Highlights meaningful words

Important Viva Questions

Basic Questions

What is tokenization?

Splitting text into tokens.

What are stopwords?

Common less-important words.

What is stemming?

Reducing words to root form.

What is lemmatization?

Converting to meaningful dictionary root.

Difference between stemming and lemmatization?

Stemming	Lemmatization
Fast	Accurate
Rule-based	Dictionary-based

Intermediate Questions

What is POS tagging?

Assigning grammatical labels.

What is TF-IDF?

Text representation technique.

Why TF-IDF used?

To identify important words.

What is corpus?

Collection of documents.

Advanced Questions

Why IDF uses logarithm?

To reduce effect of very frequent words.

Why remove stopwords?

They do not add useful meaning.

Which words get highest TF-IDF?

Rare but important words.

External May Ask Modifications

Sentence Tokenization

```
from nltk.tokenize import sent_tokenize

sent_tokenize(document)
```

Count Word Frequency

```
from collections import Counter

Counter(tokens)
```

Bag of Words

```
from sklearn.feature_extraction.text import CountVectorizer

cv = CountVectorizer()

X = cv.fit_transform(documents)

print(X.toarray())
```

Difference Between Bag of Words and TF-IDF

Bag of Words	TF-IDF
Raw count	Weighted importance

Common Exam Errors

Error	Reason
punkt not found	nltk.download missing
stopwords error	stopwords corpus not downloaded
Wrong indentation	Python loop issue

2-Minute Viva Explanation

“In this practical, we performed various NLP preprocessing techniques on a sample document using Python NLTK library. First, we tokenized the document into words, then applied POS tagging to identify grammatical categories. We removed stopwords to eliminate less meaningful words. Next, we applied stemming and lemmatization to reduce words to root forms. Finally, we represented the documents numerically using TF-IDF, which measures both term frequency and inverse document frequency to identify important words in documents.”

This output is from:

```
TfidfVectorizer()
```

It shows:

1. Vocabulary words (features)
2. TF-IDF matrix values

Let's understand it properly for viva.

Step 1 — Feature Names

```
['analytics' 'for' 'interesting' 'is' 'language' 'learning'
 'machine' 'natural' 'processing' 'text' 'useful' 'uses']
```

These are:

unique words extracted from all documents

This is called:

`Vocabulary`

Suppose Documents Were

1. "Natural language processing is interesting"
2. "Text analytics uses natural language processing"
3. "Machine learning is useful for analytics"

What Vectorizer Did

It:

1. Broke sentences into words
2. Removed duplicates
3. Sorted words alphabetically

Final vocabulary became:

Index	Word
0	analytics
1	for
2	interesting
3	is
4	language
5	learning
6	machine
7	natural
8	processing
9	text
10	useful
11	uses

Step 2 — Understanding Matrix

Matrix:

```
[[0.         0.         0.54935123 ... ]
 [0.36617957 0.         0.         ... ]
 [0.3349067  0.44036207 0.         ... ]]
```

Rows represent:

documents

Columns represent:

vocabulary words

Matrix Structure

Row	Document
Row 1	Document 1

Row	Document
Row 2	Document 2
Row 3	Document 3

Column Mapping

Column	Word
0	analytics
1	for
2	interesting
3	is
4	language
5	learning
6	machine
7	natural
8	processing
9	text
10	useful
11	uses

Understanding First Row

First row:

```
[0. 0. 0.54935123 0.41779577 0.41779577 0.
 0. 0.41779577 0.41779577 0. 0. 0.]
```

Represents:

`Document 1`
Document 1:

```
"Natural language processing is interesting"
```

Non-Zero Values Mean Word Exists

For example:

`0.54935123`
is under column:

`interesting`
Meaning:

- Word “interesting” exists in document 1
- TF-IDF score = 0.54935123

Why Some Values Are Zero?

Because those words do NOT appear in that document.

Example:

`machine`
does not exist in document 1.

So value is:

`0`

Important Interpretation

Higher TF-IDF score means:

word is more important in that document

Example

In document 1:

`interesting`
has highest score:

`0.54935123`
because:

- It appears in document 1

- Rare in other documents

So it becomes important.

Why Common Words Get Lower Scores

Words like:

`is`
`language`
`natural`
`processing`
appear in multiple documents.

Therefore:

- IDF decreases
- TF-IDF score becomes smaller

Second Row Explanation

Second row:

```
[0.36617957 0. 0. 0. 0.36617957 0.
 0. 0.36617957 0.36617957 0.48148213 0. 0.48148213]
```

Represents:

`Document 2`
Document:

`"Text analytics uses natural language processing"`

Highest Scores

Words:

`text`
`uses`
have highest TF-IDF because:

- They are unique
- Rare in other documents

Third Row Explanation

Third row:

```
[0.3349067 0.44036207 0. 0.3349067 ...]
```

Represents:

```
Document 3
```

Document:

```
"Machine learning is useful for analytics"
```

Highest Scores

Words:

```
machine
learning
useful
```

because they appear uniquely in document 3.

Important Viva Explanation

You can say:

> "Rows represent documents and columns represent unique words called vocabulary. Each value in matrix is TF-IDF score representing importance of a word in a document. Higher value means word is more important in that document."

VERY IMPORTANT Viva Questions

Q. Why some values are zero?

Because those words are absent in that document.

Q. Why different TF-IDF scores?

Because TF-IDF depends on:

- Frequency inside document
- Rarity across documents

Q. Why unique words get high TF-IDF?

Because IDF becomes high for rare words.

Q. What does row represent?

Document.

Q. What does column represent?

Vocabulary word.

Easy Memory Trick

Matrix Part	Meaning
Rows	Documents
Columns	Words
Values	TF-IDF scores

Most Important Theory Line

> "TF measures local importance inside a document, while IDF measures global rarity across all documents."